VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: FCFS and SJF Scheduling algorithms

Experiment No: 1 **Date: 23-07-2026**

Aim: write a c-program for FCFS scheduling algorithm A. without arrival times, B. with arrival times

A. without arrival times

Algorithm:

1. Start.
2. Input the number of processes (n).
3. Input the Arrival Time (AT) and Burst Time (BT) for each process.
4. Initialize current_time = 0, total_wt = 0, and total_tat = 0.
5. For each process:
    - If current_time < AT, set current_time = AT.
    - Calculate CT = current_time + BT.
    - Calculate TAT = CT − AT.
    - Calculate WT = TAT − BT.
    - Update current_time = CT.
6. Add WT and TAT to their respective totals.
    - Calculate:
    - Average Waiting Time = Total WT / n
    - Average Turnaround Time = Total TAT / n
7. Display AT, BT, CT, WT, TAT, Average WT, and Average TAT.
8. Stop.

B. with arrival times

Algorithm:

1. Start.
2. Input the number of processes (n).
3. Input the Burst Time (BT) for each process.
4. Initialize WT[0] = 0, total_wt = 0, and total_tat = 0.
5. For each process:
    - Calculate WT using the previous process's burst time.
    - Calculate TAT = WT + BT.
    - Add WT and TAT to their respective totals.
6. Calculate:
    - Average Waiting Time = Total WT / n
    - Average Turnaround Time = Total TAT / n
7. Display BT, WT, TAT, Average WT, and Average TAT.
8. Stop.

A. without arrival times

Program:

```
#include <stdio.h>
int main() {
   int n, bt[20], wt[20], tat[20];
   float twt = 0, ttat = 0;
   printf("Roll no: 25071A6255\n");
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Burst Time for P%d: ", i + 1);
      scanf("%d", &bt[i]);
   }
   wt[0] = 0;
   for (int i = 0; i < n; i++) {
      if (i > 0) wt[i] = wt[i - 1] + bt[i - 1];
      tat[i] = wt[i] + bt[i];
      twt += wt[i]; ttat += tat[i];
   }
   printf("\nProcess\tBT\tWT\tTAT\n");
   for (int i = 0; i < n; i++)
      printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
   printf("\nAvg WT = %.2f | Avg TAT = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll No: 25071a6255
Enter number of processes: 3
Enter Burst Time for process 1: 24
Enter Burst Time for process 2: 3
Enter Burst Time for process 3: 3

Process Burst Time      Waiting Time    Turnaround Time
P1              24              0               24
P2              3               24              27
P3              3               27              30

Average Waiting Time: 17.00
Average Turnaround Time: 27.00
```

```
Roll No: 25071a6255
Enter number of processes: 4
Enter Burst Time for process 1: 2
Enter Burst Time for process 2: 10
Enter Burst Time for process 3: 4
Enter Burst Time for process 4: 7

Process Burst Time      Waiting Time    Turnaround Time
P1              2               0               2
P2              10              2               12
P3              4               12              16
P4              7               16              23

Average Waiting Time: 7.50
Average Turnaround Time: 13.25
```

```
Roll No: 25071a6255
Enter number of processes: 4
Enter Burst Time for process 1: 2
Enter Burst Time for process 2: 4
Enter Burst Time for process 3: 6
Enter Burst Time for process 4: 8

Process Burst Time      Waiting Time    Turnaround Time
P1              2               0               2
P2              4               2               6
P3              6               6               12
P4              8               12              20

Average Waiting Time: 5.00
Average Turnaround Time: 10.00
```

B. with arrival times

Program:

```
#include <stdio.h>
int main() {
   int n, at[20], bt[20], time = 0;
   float twt = 0, ttat = 0;
   printf("Roll no: 25071A6255\n");
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]);
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      if (time < at[i]) time = at[i];
      time += bt[i];
      int tat = time - at[i], wt = tat - bt[i];
      twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], time, wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 3

Process AT      BT      CT      WT      TAT
P1      0       24      24      0       24
P2      1       3       27      23      26
P3      2       3       30      25      28
Average Waiting Time = 16.00
Average Turnaround Time = 26.00
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 1 4
Enter Arrival Time and Burst Time for P2: 2 3
Enter Arrival Time and Burst Time for P3: 10 2

Process AT      BT      CT      WT      TAT
P1      1       4       5       0       4
P2      2       3       8       3       6
P3      10      2       12      0       2
Average Waiting Time = 1.00
Average Turnaround Time = 4.00
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      24      0       24
P2      0       3       27      24      27
P3      0       3       30      27      30
Average Waiting Time = 17.00
Average Turnaround Time = 27.00
```

Aim: write a c-program for SJF scheduling algorithm A. Non-preemptive B. Preemptive

A. non-preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT).
3. Initialize current_time = 0 and mark all processes as unvisited.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ current_time) with the smallest Burst Time (BT).
    - If no process has arrived, increase current_time by 1.
5. Otherwise:
    - Execute the selected process.
    - Calculate CT, TAT, and WT.
    - Update current_time = CT.
    - Mark the process as completed.
6. Calculate the Average Waiting Time (AWT) and Average Turnaround Time (ATAT).
7. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
8. Stop.

B. Preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ time) with the smallest Remaining Time (RT).
    - If no process is ready, increase time by 1.
    - Otherwise:
        - Execute the selected process for 1 time unit.
        - Decrease RT by 1 and increase time by 1.
        - If RT = 0, calculate CT, mark the process as completed, and increment completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
7. Stop.

A. non-preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, p[10], at[10], bt[10], ct[10], tat[10], wt[10], vis[10]={0}, cur=0, done=0;
   float twt=0, ttat=0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for(int i=0; i<n; i++) { p[i]=i+1; printf("Enter Arrival Time and Burst Time for P%d: ", i+1); scanf("%d%d",
&at[i], &bt[i]); }
   while(done < n) {
      int idx = -1, min_bt = 1e9;
      for(int i=0; i<n; i++)
         if(at[i] <= cur && !vis[i] && bt[i] < min_bt) { min_bt = bt[i]; idx = i; }
      if(idx == -1) cur++;
      else {
         cur += bt[idx]; ct[idx] = cur; tat[idx] = ct[idx] - at[idx];
         wt[idx] = tat[idx] - bt[idx]; twt += wt[idx]; ttat += tat[idx]; vis[idx] = 1; done++;
      }
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for(int i=0; i<n; i++) printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], at[i], bt[i], ct[i], wt[i], tat[i]);
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt/n, ttat/n);
   return 0;}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 5 6
Enter Arrival Time and Burst Time for P3: 6 2

Process AT      BT      CT      WT      TAT
P1      0       3       3       0       3
P2      5       6       11      0       6
P3      6       2       13      5       7

Average Waiting Time = 1.67
Average Turnaround Time = 5.33
```

```
Roll no: 25071A6255
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 7
Enter Arrival Time and Burst Time for P2: 2 4
Enter Arrival Time and Burst Time for P3: 4 1
Enter Arrival Time and Burst Time for P4: 5 4

Process AT      BT      CT      WT      TAT
P1      0       7       7       0       7
P2      2       4       12      6       10
P3      4       1       8       3       4
P4      5       4       16      7       11

Average Waiting Time = 4.00
Average Turnaround Time = 8.00
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       3       0       3
P3      0       3       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

B. preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, at[10], bt[10], rt[10], ct[10], done = 0, time = 0, min_rt, idx;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i];
   }
   while (done < n) {
      min_rt = 1e9; idx = -1;
      for (int i = 0; i < n; i++)
         if (at[i] <= time && rt[i] > 0 && rt[i] < min_rt) { min_rt = rt[i]; idx = i; }
      if (idx == -1) { time++; continue; }
      rt[idx]--; time++;
      if (rt[idx] == 0) { done++; ct[idx] = time; }
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i];
      twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 2

Process AT      BT      CT      WT      TAT
P1      0       8       14      6       14
P2      1       4       7       2       6
P3      2       2       4       0       2

Average Waiting Time = 2.67
Average Turnaround Time = 7.33
```

```
Roll no: 25071A6255
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 9
Enter Arrival Time and Burst Time for P4: 3 5

Process AT      BT      CT      WT      TAT
P1      0       8       17      9       17
P2      1       4       5       0       4
P3      2       9       26      15      24
P4      3       5       10      2       7

Average Waiting Time = 6.50
Average Turnaround Time = 13.00
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       3       0       3
P3      0       3       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Aim: write a c-program for priority scheduling algorithm A. Non-preemptive B. Preemptive

A. non-preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), Burst Time (BT), and Priority (Pr).
3. Initialize current_time = 0 and mark all processes as unvisited.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ current_time) with the highest priority (lowest Priority value).
    - If no process is ready, increase current_time by 1.
    - Otherwise:
        - Execute the selected process.
        - Calculate CT, TAT, and WT.
        - Update current_time = CT.
        - Mark the process as completed.
5. Calculate Average Waiting Time (AWT) and Average Turnaround Time (ATAT).
6. Display AT, BT, Priority, CT, WT, TAT, AWT, and ATAT.
7. Stop.

B. Preemptive

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), Burst Time (BT), and Priority (Pr). Set Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the arrived process (AT ≤ time) with the highest priority (lowest Priority value).
    - If no process is ready, increase time by 1.
    - Otherwise:
        - Execute the selected process for 1 time unit.
        - Decrease RT by 1 and increase time by 1.
        - If RT = 0, calculate CT, mark the process as completed, and increment completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, Priority, CT, WT, TAT, AWT, and ATAT.
7. Stop.

A. non-preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, at[10], bt[10], pr[10], ct[10], vis[10]={0}, cur=0, done=0;
   float twt=0, ttat=0;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for(int i=0; i<n; i++) {
 printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i+1); scanf("%d%d%d", &at[i], &bt[i], &pr[i]); }
   while(done < n) {
      int idx = -1, min_pr = 1e9;
      for(int i=0; i<n; i++)
         if(at[i] <= cur && !vis[i] && pr[i] < min_pr) { min_pr = pr[i]; idx = i; }
      if(idx == -1) cur++;
      else { cur += bt[idx]; ct[idx] = cur; vis[idx] = 1; done++; }
   }
   printf("\nProcess\tAT\tBT\tPr\tCT\tWT\tTAT\n");
   for(int i=0; i<n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], pr[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt/n, ttat/n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 10 3
Enter Arrival Time, Burst Time and Priority for P2: 1 1 1
Enter Arrival Time, Burst Time and Priority for P3: 2 2 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       10      3       10      0       10
P2      1       1       1       11      9       10
P3      2       2       2       13      9       11

Average Waiting Time = 6.00
Average Turnaround Time = 10.33
```

```
Roll no: 25071A6255
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority for P1: 0 11 2
Enter Arrival Time, Burst Time and Priority for P2: 1 3 1
Enter Arrival Time, Burst Time and Priority for P3: 2 5 4
Enter Arrival Time, Burst Time and Priority for P4: 3 2 3

Process AT      BT      Pr      CT      WT      TAT
P1      0       11      2       11      0       11
P2      1       3       1       14      10      13
P3      2       5       4       21      14      19
P4      3       2       3       16      11      13

Average Waiting Time = 8.75
Average Turnaround Time = 14.00
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       24      3       30      6       30
P2      0       3       1       3       0       3
P3      0       3       2       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

B. preemptive

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, at[10], bt[10], pr[10], rt[10], ct[10], done = 0, time = 0, min_pr, idx;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time, Burst Time and Priority for P%d: ", i + 1);
      scanf("%d%d%d", &at[i], &bt[i], &pr[i]); rt[i] = bt[i];
   }
   while (done < n) {
      min_pr = 1e9; idx = -1;
      for (int i = 0; i < n; i++)
         if (at[i] <= time && rt[i] > 0 && pr[i] < min_pr) { min_pr = pr[i]; idx = i; }
      if (idx == -1) { time++; continue; }
      rt[idx]--; time++;
      if (rt[idx] == 0) { done++; ct[idx] = time; }
   }
   printf("\nProcess\tAT\tBT\tPr\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], pr[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 7 3
Enter Arrival Time, Burst Time and Priority for P2: 1 4 1
Enter Arrival Time, Burst Time and Priority for P3: 2 1 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       7       3       12      5       12
P2      1       4       1       5       0       4
P3      2       1       2       6       3       4

Average Waiting Time = 2.67
Average Turnaround Time = 6.67
```

```
Roll no: 25071A6255
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority for P1: 0 10 3
Enter Arrival Time, Burst Time and Priority for P2: 1 1 1
Enter Arrival Time, Burst Time and Priority for P3: 2 2 4
Enter Arrival Time, Burst Time and Priority for P4: 3 1 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       10      3       12      2       12
P2      1       1       1       2       0       1
P3      2       2       4       14      10      12
P4      3       1       2       4       0       1

Average Waiting Time = 3.00
Average Turnaround Time = 6.50
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       24      3       30      6       30
P2      0       3       1       3       0       3
P3      0       3       2       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Aim: write a c-program for Round Robin scheduling algorithm A. Time Quantum=2, B. Time Quantum=10

A. Time Quantum=2

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Time Quantum (TQ) = 2 and Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the first process from the Ready Queue.
    - Execute the process for 2 units or until it finishes.
    - Update RT and time.
    - Add any newly arrived processes to the Ready Queue.
    - If RT > 0, add the process back to the Ready Queue; otherwise, calculate CT and mark it as completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
7. Stop.

B. Time Quantum=10

Algorithm:

1. Start.
2. Input the number of processes (n), Arrival Time (AT), and Burst Time (BT). Set Time Quantum (TQ) = 10 and Remaining Time (RT) = BT for each process.
3. Initialize time = 0 and completed = 0.
4. Repeat until all processes are completed:
    - Select the first process from the Ready Queue.
    - Execute the process for 10 units or until it finishes.
    - Update RT and time.
    - Add any newly arrived processes to the Ready Queue.
    - If RT > 0, add the process back to the Ready Queue; otherwise, calculate CT and mark it as completed.
5. Calculate TAT, WT, Average Waiting Time (AWT), and Average Turnaround Time (ATAT).
6. Display AT, BT, CT, WT, TAT, AWT, and ATAT.
7. Stop.

A. Time Quantum=2

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, tq = 2, at[10], bt[10], rt[10], ct[10], done = 0, time = 0;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i];}
   while (done < n) {
      int exec = 0;
      for (int i = 0; i < n; i++) {
         if (at[i] <= time && rt[i] > 0) {
            exec = 1;
            int slice = (rt[i] > tq) ? tq : rt[i];
            rt[i] -= slice; time += slice;
            if (rt[i] == 0) { done++; ct[i] = time; } }}
      if (!exec) time++;}
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat); }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 3
Enter time quantum: 2
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 1

Process AT      BT      CT      WT      TAT
P1      0       5       9       4       9
P2      1       3       8       4       7
P3      2       1       5       2       3

Average Waiting Time = 3.33
Average Turnaround Time = 6.33
```

```
Roll no: 25071A6255
Enter number of processes: 4
Enter time quantum: 2
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 2
Enter Arrival Time and Burst Time for P4: 4 1

Process AT      BT      CT      WT      TAT
P1      0       5       12      7       12
P2      1       4       11      6       10
P3      2       2       6       2       4
P4      4       1       7       2       3

Average Waiting Time = 4.25
Average Turnaround Time = 7.25
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter time quantum: 2
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       9       6       9
P3      0       3       10      7       10

Average Waiting Time = 6.33
Average Turnaround Time = 16.33
```

B. Time Quantum=10

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, tq = 10, at[10], bt[10], rt[10], ct[10], done = 0, time = 0;
   float twt = 0, ttat = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]); rt[i] = bt[i];
   }
   while (done < n) {
      int exec = 0;
      for (int i = 0; i < n; i++) {
         if (at[i] <= time && rt[i] > 0) {
            exec = 1;
            int slice = (rt[i] > tq) ? tq : rt[i];
            rt[i] -= slice; time += slice;
            if (rt[i] == 0) { done++; ct[i] = time; }
         }
      }
      if (!exec) time++;
   }
   printf("\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
   for (int i = 0; i < n; i++) {
      int tat = ct[i] - at[i], wt = tat - bt[i]; twt += wt; ttat += tat;
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], ct[i], wt, tat);
   }
   printf("\nAverage Waiting Time = %.2f\nAverage Turnaround Time = %.2f\n", twt / n, ttat / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 3
Enter time quantum: 10
Enter Arrival Time and Burst Time for P1: 0 25
Enter Arrival Time and Burst Time for P2: 2 6
Enter Arrival Time and Burst Time for P3: 4 12

Process AT      BT      CT      WT      TAT
P1      0       25      43      18      43
P2      2       6       16      8       14
P3      4       12      38      22      34

Average Waiting Time = 16.00
Average Turnaround Time = 30.33
```

```
Roll no: 25071A6255
Enter number of processes: 4
Enter time quantum: 10
Enter Arrival Time and Burst Time for P1: 0 15
Enter Arrival Time and Burst Time for P2: 1 8
Enter Arrival Time and Burst Time for P3: 3 22
Enter Arrival Time and Burst Time for P4: 5 5

Process AT      BT      CT      WT      TAT
P1      0       15      38      23      38
P2      1       8       18      9       17
P3      3       22      50      25      47
P4      5       5       33      23      28

Average Waiting Time = 20.00
Average Turnaround Time = 32.50
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter time quantum: 10
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       13      10      13
P3      0       3       16      13      16

Average Waiting Time = 9.67
Average Turnaround Time = 19.67
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: Banker's algorithm and Analysis of the scheduling algorithms

Experiment No: 3 **Date:** 23-07-2026

Aim: Make the analysis of all scheduling algorithms

Testcases:

```
Roll No: 25071a6255
Enter number of processes: 3
Enter Burst Time for process 1: 24
Enter Burst Time for process 2: 3
Enter Burst Time for process 3: 3

Process Burst Time      Waiting Time    Turnaround Time
P1              24              0               24
P2              3               24              27
P3              3               27              30

Average Waiting Time: 17.00
Average Turnaround Time: 27.00
```

FCFS scheduling without arrival times

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 1 4
Enter Arrival Time and Burst Time for P2: 2 3
Enter Arrival Time and Burst Time for P3: 10 2

Process AT      BT      CT      WT      TAT
P1      1       4       5       0       4
P2      2       3       8       3       6
P3      10      2       12      0       2
Average Waiting Time = 1.00
Average Turnaround Time = 4.00
```

FCFS scheduling arrival times

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time and Burst Time for P1: 0 3
Enter Arrival Time and Burst Time for P2: 5 6
Enter Arrival Time and Burst Time for P3: 6 2

Process AT      BT      CT      WT      TAT
P1      0       3       3       0       3
P2      5       6       11      0       6
P3      6       2       13      5       7

Average Waiting Time = 1.67
Average Turnaround Time = 5.33
```

SJF scheduling non-preemptive

```
Roll no: 25071A6255
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 8
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 9
Enter Arrival Time and Burst Time for P4: 3 5

Process AT      BT      CT      WT      TAT
P1      0       8       17      9       17
P2      1       4       5       0       4
P3      2       9       26      15      24
P4      3       5       10      2       7

Average Waiting Time = 6.50
Average Turnaround Time = 13.00
```

SJF scheduling preemptive

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 10 3
Enter Arrival Time, Burst Time and Priority for P2: 1 1 1
Enter Arrival Time, Burst Time and Priority for P3: 2 2 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       10      3       10      0       10
P2      1       1       1       11      9       10
P3      2       2       2       13      9       11

Average Waiting Time = 6.00
Average Turnaround Time = 10.33
```

Priority scheduling non-preemptive

```
Roll no: 25071A6255
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for P1: 0 24 3
Enter Arrival Time, Burst Time and Priority for P2: 0 3 1
Enter Arrival Time, Burst Time and Priority for P3: 0 3 2

Process AT      BT      Pr      CT      WT      TAT
P1      0       24      3       30      6       30
P2      0       3       1       3       0       3
P3      0       3       2       6       3       6

Average Waiting Time = 3.00
Average Turnaround Time = 13.00
```

Priority scheduling preemptive

```
Roll no: 25071A6255
Enter number of processes: 3
Enter time quantum: 2
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       9       6       9
P3      0       3       10      7       10

Average Waiting Time = 6.33
Average Turnaround Time = 16.33
```

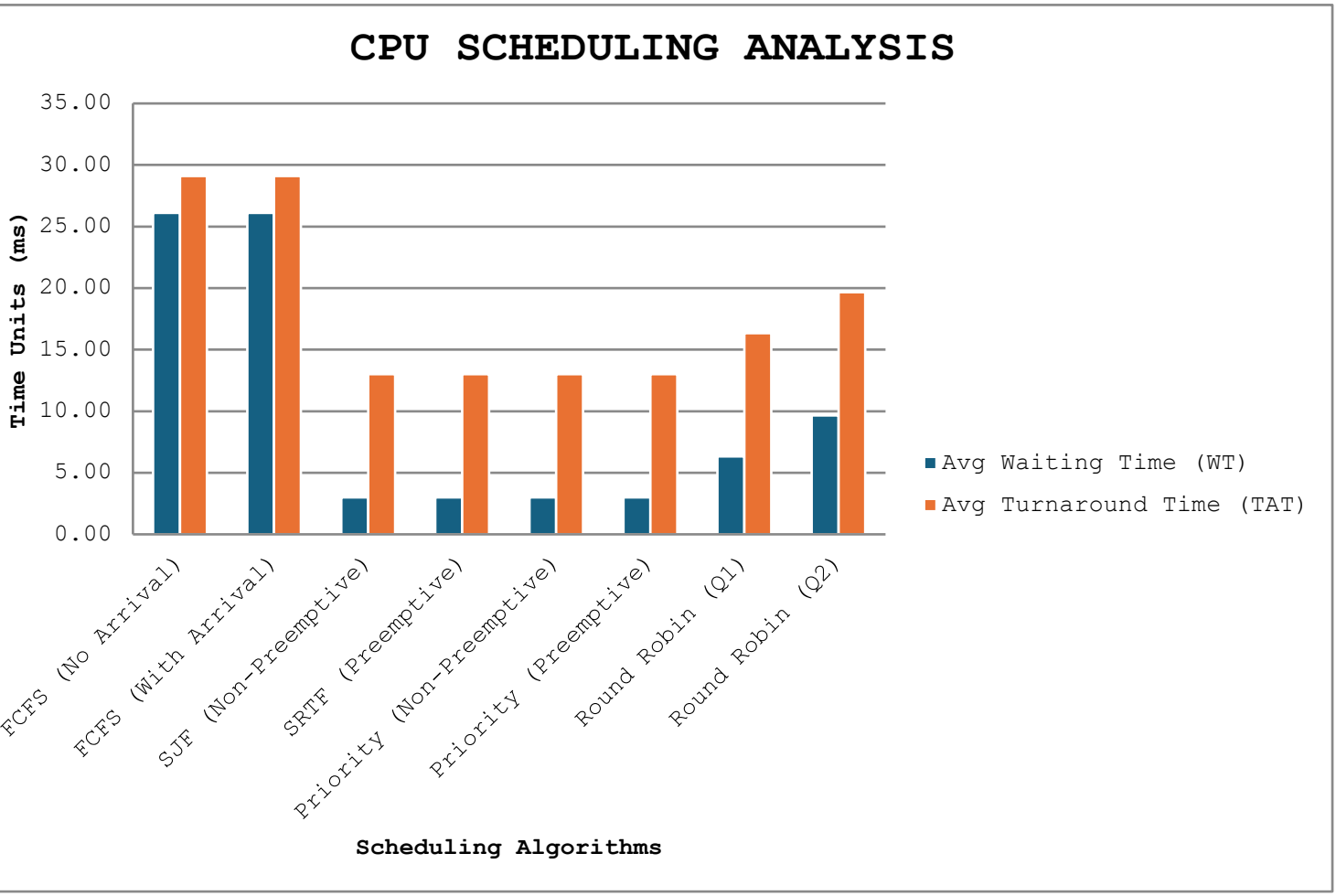
Round Robin scheduling, Time quantum=2

```
Roll no: 25071A6255
Enter number of processes: 3
Enter time quantum: 2
Enter Arrival Time and Burst Time for P1: 0 24
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 0 3

Process AT      BT      CT      WT      TAT
P1      0       24      30      6       30
P2      0       3       9       6       9
P3      0       3       10      7       10

Average Waiting Time = 6.33
Average Turnaround Time = 16.33
```

Round Robin scheduling, Time quantum=10


CPU SCHEDULING ANALYSIS
35.00
30.00
25.00
20.00
15.00
10.00
5.00
0.00
Time Units (ms)
FCFS (No Arrival)
FCFS (With Arrival)
SJF (Non-Preemptive)
SRTF (Preemptive)
Priority (Non-Preemptive)
Priority (Preemptive)
Round Robin (Q1)
Round Robin (Q2)
Scheduling Algorithms
Avg Waiting Time (WT)
Avg Turnaround Time (TAT)

Aim: write a c-program for Banker's algorithm

Algorithm:

1. **Start.**
2. Input the number of processes (n), resource types (m), Allocation, Max, and Available matrices.
3. Calculate the Need Matrix:
    - Need = Max − Allocation
4. Initialize:
    - Work = Available
    - Mark all processes as unfinished.
5. Repeat until all processes are finished:
    - Select an unfinished process whose Need ≤ Work.
    - If found:
        - Work = Work + Allocation
        - Mark the process as finished.
        - Add it to the Safe Sequence.
    - If no suitable process is found, stop.
6. Check the result:
    - If all processes are finished, the system is in a Safe State and display the Safe Sequence.
    - Otherwise, the system is in an Unsafe State.
7. **Stop**.

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, m, alloc[10][10], max[10][10], need[10][10], avail[10], finish[10] = {0}, safeSeq[10], count = 0;
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter number of resource types: "); scanf("%d", &m);
   printf("Enter Allocation Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
   printf("Enter Max Matrix:\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < m; j++) {
         scanf("%d", &max[i][j]);
         need[i][j] = max[i][j] - alloc[i][j];
      }
   printf("Enter Available Resources:\n");
   for (int j = 0; j < m; j++) scanf("%d", &avail[j]);
   int work[10];
   for (int j = 0; j < m; j++) work[j] = avail[j];
  while (count < n) {
      int found = 0;
      for (int i = 0; i < n; i++) {
         if (!finish[i]) {
            int possible = 1;
            for (int j = 0; j < m; j++) {
               if (need[i][j] > work[j]) { possible = 0; break; }
            }
            if (possible) {
               for (int j = 0; j < m; j++) work[j] += alloc[i][j];
               safeSeq[count++] = i;
```

```
            finish[i] = 1;
            found = 1; }
        }
    }
    if (!found) break;
  }
  if (count == n) {
    printf("\nSystem is in a SAFE STATE.\nSafe Sequence: ");
    for (int i = 0; i < n; i++) printf("P%d ", safeSeq[i] + 1);
    printf("\n");
  } else {
    printf("\nSystem is in an UNSAFE STATE (Deadlock possible).\n");
  }
  return 0;}
```

Testcases:

```
Roll no: 25071A6255
Enter number of processes: 5
Enter number of resource types: 4
Enter Allocation Matrix:
0 1 1 0
1 4 4 1
1 3 6 5
0 6 3 2
0 0 1 4
Enter Max Matrix:
0 2 1 0
1 6 5 2
2 3 6 6
0 6 5 2
0 6 5 6
Enter Available Resources:
1 3 1 0

System is in a SAFE STATE.
Safe Sequence: P1 P4 P5 P2 P3

Process returned 0 (0x0)   execution time : 86.140 s
Press any key to continue.
```

```
Roll no: 25071A6255
Enter number of processes: 3
Enter number of resource types: 3
Enter Allocation Matrix:
1 0 0
0 1 0
0 0 1
Enter Max Matrix:
2 1 1
1 2 1
1 1 2
Enter Available Resources:
0 0 0

System is in an UNSAFE STATE (Deadlock possible).

Process returned 0 (0x0)   execution time : 30.658 s
Press any key to continue.
```

```
Roll no: 25071A6255
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a SAFE STATE.
Safe Sequence: P2 P4 P5 P1 P3

Process returned 0 (0x0)   execution time : 45.122 s
Press any key to continue.
```

Aim: Implementation of the following Memory Allocation Methods for fixed partition a) First Fit b) Worst Fit c) Best Fit

A. First Fit

Algorithm:

1. Start.
2. Input the number of memory partitions (m) and their sizes, and the number of processes (n) and their memory requirements.
3. Initialize all processes as unallocated.
4. For each process:
   - Search the partitions from the first partition.
   - If an unallocated partition is large enough, allocate it to the process.
   - Mark the partition as allocated and stop searching.
5. Calculate the Internal Fragmentation for each allocated partition:
   - Fragmentation = Partition Size − Process Size
6. Display the process, allocated partition, partition size, and internal fragmentation.
7. Stop.

B. Worst Fit

Algorithm:

1. Start.
2. Input the number of memory partitions (m) and their sizes, and the number of processes (n) and their memory requirements.
3. Initialize all processes as unallocated.
4. For each process:
   - Search all unallocated partitions.
   - Find the largest partition that is large enough for the process.
   - If a suitable partition is found, allocate it to the process and mark it as allocated.
5. Calculate the Internal Fragmentation:
   - Fragmentation = Partition Size − Process Size
6. Display the process, allocated partition, partition size, and internal fragmentation.
7. Stop.

C. Best Fit

Algorithm:

1. Start.
2. Input the number of memory partitions (m) and their sizes, and the number of processes (n) and their memory requirements.
3. Initialize all processes as unallocated.
4. For each process:
   - Search all unallocated partitions.
   - Find the smallest partition that is large enough for the process.
   - If a suitable partition is found, allocate it to the process and mark it as allocated.
5. Calculate the Internal Fragmentation:
   - Fragmentation = Partition Size − Process Size
6. Display the process, allocated partition, partition size, and internal fragmentation.
7. Stop.

A. First Fit

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int m, n, bsize[10], psize[10], alloc[10], occupied[10] = {0};
   printf("Enter number of blocks: "); scanf("%d", &m);
   printf("Enter block sizes: ");
   for (int i = 0; i < m; i++) scanf("%d", &bsize[i]);
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter process sizes: ");
   for (int i = 0; i < n; i++) {
    scanf("%d", &psize[i]); alloc[i] = -1;
 }
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
         if (!occupied[j] && bsize[j] >= psize[i]) {
            alloc[i] = j;
            occupied[j] = 1;
            break; }  }    }
   printf("\nProcess\tProcess Size\tBlock No.\tBlock Size\tInternal Frag\n");
   for (int i = 0; i < n; i++) {
      if (alloc[i] != -1)
         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, psize[i], alloc[i] + 1, bsize[alloc[i]], bsize[alloc[i]] - psize[i]);
      else
         printf("P%d\t%d\t\tNot Allocated\t-\t\t-\n", i + 1, psize[i]);}
   return 0;}
```

Testcases:

```
Roll no: 25071A6255
Enter number of blocks: 4
Enter block sizes: 200 400 600 500
Enter number of processes: 4
Enter process sizes: 150 350 450 100

Process Process Size    Block No.       Block Size      Internal Frag
P1      150             1               200             50
P2      350             2               400             50
P3      450             3               600             150
P4      100             4               500             400
```

```
Roll no: 25071A6255
Enter number of blocks: 5
Enter block sizes: 300 150 200 350 600
Enter number of processes: 4
Enter process sizes: 120 290 180 550

Process Process Size    Block No.       Block Size      Internal Frag
P1      120             1               300             180
P2      290             4               350             60
P3      180             3               200             20
P4      550             5               600             50
```

B. Worst Fit

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int m, n, bsize[10], psize[10], alloc[10], occupied[10] = {0};
   printf("Enter number of blocks: "); scanf("%d", &m);
   printf("Enter block sizes: ");
   for (int i = 0; i < m; i++) scanf("%d", &bsize[i]);
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter process sizes: ");
   for (int i = 0; i < n; i++) { scanf("%d", &psize[i]); alloc[i] = -1; }
   for (int i = 0; i < n; i++) {
      int worstIdx = -1;
      for (int j = 0; j < m; j++) {
         if (!occupied[j] && bsize[j] >= psize[i]) {
            if (worstIdx == -1 || bsize[j] > bsize[worstIdx])
               worstIdx = j;}    }
      if (worstIdx != -1) {
         alloc[i] = worstIdx;
         occupied[worstIdx] = 1; }}
   printf("\nProcess\tProcess Size\tBlock No.\tBlock Size\tInternal Frag\n");
   for (int i = 0; i < n; i++) {
      if (alloc[i] != -1)
         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, psize[i], alloc[i] + 1, bsize[alloc[i]], bsize[alloc[i]] - psize[i]);
      else
         printf("P%d\t%d\t\tNot Allocated\t-\t\t-\n", i + 1, psize[i]); }
   return 0;}
```

Testcases:

```
Roll no: 25071A6255
Enter number of blocks: 5
Enter block sizes: 100 500 200 300 600
Enter number of processes: 4
Enter process sizes: 212 417 112 426

Process Process Size    Block No.       Block Size      Internal Frag
P1      212             5               600             388
P2      417             2               500             83
P3      112             4               300             188
P4      426             Not Allocated   -               -
```

```
Roll no: 25071A6255
Enter number of blocks: 4
Enter block sizes: 200 400 600 500
Enter number of processes: 4
Enter process sizes: 150 250 100 300

Process Process Size    Block No.       Block Size      Internal Frag
P1      150             3               600             450
P2      250             4               500             250
P3      100             2               400             300
P4      300             Not Allocated   -               -
```

```
Roll no: 25071A6255
Enter number of blocks: 5
Enter block sizes: 300 150 200 350 600
Enter number of processes: 4
Enter process sizes: 120 290 180 100

Process Process Size    Block No.       Block Size      Internal Frag
P1      120             5               600             480
P2      290             4               350             60
P3      180             1               300             120
P4      100             3               200             100
```

C. Best Fit

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int m, n, bsize[10], psize[10], alloc[10], occupied[10] = {0};
   printf("Enter number of blocks: "); scanf("%d", &m);
   printf("Enter block sizes: ");
   for (int i = 0; i < m; i++) scanf("%d", &bsize[i]);
   printf("Enter number of processes: "); scanf("%d", &n);
   printf("Enter process sizes: ");
   for (int i = 0; i < n; i++) { scanf("%d", &psize[i]); alloc[i] = -1; }

   for (int i = 0; i < n; i++) {
      int bestIdx = -1;
      for (int j = 0; j < m; j++) {
         if (!occupied[j] && bsize[j] >= psize[i]) {
            if (bestIdx == -1 || bsize[j] < bsize[bestIdx])
               bestIdx = j;
         }
      }
      if (bestIdx != -1) {
         alloc[i] = bestIdx;
         occupied[bestIdx] = 1;
      }
   }
   printf("\nProcess\tProcess Size\tBlock No.\tBlock Size\tInternal Frag\n");
   for (int i = 0; i < n; i++) {
      if (alloc[i] != -1)
         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, psize[i], alloc[i] + 1, bsize[alloc[i]], bsize[alloc[i]] - psize[i]);
      else
         printf("P%d\t%d\t\tNot Allocated\t-\t\t-\n", i + 1, psize[i]);
   }
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of blocks: 5
Enter block sizes: 100 500 200 300 600
Enter number of processes: 4
Enter process sizes: 212 17 112 426

Process Process Size    Block No.       Block Size      Internal Frag
P1      212             4               300             88
P2      17              1               100             83
P3      112             3               200             88
P4      426             2               500             74
```

```
Roll no: 25071A6255
Enter number of blocks: 5
Enter block sizes: 300 150 200 350 600
Enter number of processes: 4
Enter process sizes: 120 290 180 550

Process Process Size    Block No.       Block Size      Internal Frag
P1      120             2               150             30
P2      290             1               300             10
P3      180             3               200             20
P4      550             5               600             50
```

```
Roll no: 25071A6255
Enter number of blocks: 4
Enter block sizes: 200 400 600 500
Enter number of processes: 4
Enter process sizes: 150 350 450 100

Process Process Size    Block No.       Block Size      Internal Frag
P1      150             1               200             50
P2      350             2               400             50
P3      450             4               500             50
P4      100             3               600             500
```

Aim: Implementation of Paging Technique of Memory Management

Algorithm:

1. Start.
2. Input the main memory size (MS) and page size (PS), then calculate the Total Frames.
3. Input the process size and calculate the Total Pages required.
4. If Total Pages > Total Frames, display "Memory insufficient" and stop.
5. Input the frame number for each page and create the Page Table.
6. Input the Logical Address (LA) and calculate:
    - Page Number = LA / PS
    - Offset = LA % PS
7. If the page number is valid:
    - Find the Frame Number from the Page Table.
    - Calculate:
        - Physical Address = (Frame Number × PS) + Offset
    - Display the Page Number, Offset, Frame Number, and Physical Address.
8. Otherwise, display "Invalid Logical Address".
9. Stop

Program:

```
#include <stdio.h>

int main() {
    printf("Roll no: 25071A6255\n");
    int ms, ps, p_size, total_frames, total_pages;
    int page_table[20];
    int la, p, d, pa;

    printf("Enter main memory size (in bytes): ");
    scanf("%d", &ms);
    printf("Enter page size (in bytes): ");
    scanf("%d", &ps);

    total_frames = ms / ps;
    printf("Total frames available in memory: %d\n\n", total_frames);

    printf("Enter process size (in bytes): ");
    scanf("%d", &p_size);

    total_pages = p_size / ps;
    if (p_size % ps != 0) total_pages++;

    if (total_pages > total_frames) {
        printf("\nMemory Allocation Failed: Not enough frames available.\n");
        return 0;
    }

    printf("Total pages required for process: %d\n", total_pages);
    printf("Enter frame numbers allocated to each page:\n");
    for (int i = 0; i < total_pages; i++) {
        printf("Page %d -> Frame No: ", i);
        scanf("%d", &page_table[i]);
    }

    printf("\nEnter Logical Address to translate: ");
    scanf("%d", &la);

    p = la / ps;
    d = la % ps;

    if (p >= total_pages) {
        printf("\nInvalid Logical Address! (Page number %d exceeds allocated pages)\n", p);
    }
```

```
else {
        pa = (page_table[p] * ps) + d;
        printf("\n--- Address Translation Details ---\n");
        printf("Page Number (p)      : %d\n", p);
        printf("Page Offset (d)      : %d\n", d);
        printf("Allocated Frame No   : %d\n", page_table[p]);
        printf("Physical Address     : %d\n", pa);
    }

    return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter main memory size (in bytes): 1024
Enter page size (in bytes): 256
Total frames available in memory: 4

Enter process size (in bytes): 512
Total pages required for process: 2
Enter frame numbers allocated to each page:
Page 0 -> Frame No: 1
Page 1 -> Frame No: 3

Enter Logical Address to translate: 300

--- Address Translation Details ---
Page Number (p) : 1
Page Offset (d) : 44
Allocated Frame No : 3
Physical Address : 812
```

```
Roll no: 25071A6255
Enter main memory size (in bytes): 1000
Enter page size (in bytes): 100
Total frames available in memory: 10

Enter process size (in bytes): 250
Total pages required for process: 3
Enter frame numbers allocated to each page:
Page 0 -> Frame No: 2
Page 1 -> Frame No: 5
Page 2 -> Frame No: 7

Enter Logical Address to translate: 220

--- Address Translation Details ---
Page Number (p) : 2
Page Offset (d) : 20
Allocated Frame No : 7
Physical Address : 720
```

```
Roll no: 25071A6255
Enter main memory size (in bytes): 1024
Enter page size (in bytes): 256
Total frames available in memory: 4

Enter process size (in bytes): 512
Total pages required for process: 2
Enter frame numbers allocated to each page:
Page 0 -> Frame No: 0
Page 1 -> Frame No: 2

Enter Logical Address to translate: 600

Invalid Logical Address! (Page number 2 exceeds allocated pages)
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: frame allocation Methods.

Experiment No: 7 **Date:** 3-09-2026

Aim: Simulate frame allocation Methods. A) minimum number of frames B) equal allocation C) proportional allocation

A. minimum number of frames

Algorithm:

1. Start.
2. Input total available frames and minimum frames required per process.
3. Input number of processes and their page requirements.
4. Check whether available frames are sufficient for all processes.
5. Allocate the minimum required frames to each process.
6. Calculate and display the remaining unallocated frames.
7. Stop.

B. equal allocation

Algorithm:

1. Start.
2. Input total number of available frames and number of processes m.
3. Calculate frames allocated to each process:
   Frames per Process = Total Frames / m
4. Calculate remaining frames:
   Leftover = Total Frames % m
5. Display frames allocated to each process and the leftover frames.
6. Stop.

C. proportional allocation

Algorithm:

1. Start.
2. Input total number of frames m and number of processes n.
3. Input page requirement s[i] for each process and calculate total requirement S.
4. Calculate frames allocated to each process:
   Allocation[i] = (s[i] / S) × m
5. Calculate total allocated frames and remaining unallocated frames.
6. Display process requirements, allocated frames, and remaining frames.
7. Stop.

A. minimum number of frames

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A6255\n\n");
   int total_frames, min_per_process, n;
   int page_req[10], allocated[10];

   printf("Enter total frames in physical memory: ");
   scanf("%d", &total_frames);
   printf("Enter minimum frames required per process (by architecture): ");
   scanf("%d", &min_per_process);
   printf("Enter number of processes: ");
   scanf("%d", &n);

   int total_min_needed = n * min_per_process;
   if (total_min_needed > total_frames) {
      printf("\nAllocation Failed! Total available frames (%d) < Required minimum frames (%d).\n",
            total_frames, total_min_needed);
      return 0;
   }

   printf("\nEnter page requirements for each process:\n");
   for (int i = 0; i < n; i++) {
      printf("Process P%d: ", i + 1);
      scanf("%d", &page_req[i]);
      allocated[i] = min_per_process;
   }

   int total_allocated = total_min_needed;
   int remaining = total_frames - total_allocated;

   printf("\n--- Minimum Frame Allocation Result ---\n");
   printf("Process\tPage Req\tAllocated Frames\tStatus\n");
   for (int i = 0; i < n; i++) {
      printf("P%d\t%d\t\t%d\t\t\t%s\n",
            i + 1, page_req[i], allocated[i],
            (allocated[i] >= page_req[i]) ? "Satisfied" : "Needs Swap Space");
   }
   printf("\nTotal Frames Allocated: %d / %d", total_allocated, total_frames);
   printf("\nRemaining Unallocated Frames: %d\n", remaining);

   return 0;
}
```

Testcases:

```
Roll no: 25071A6255

Enter total frames in physical memory: 5
Enter minimum frames required per process (by architecture): 2
Enter number of processes: 3

Allocation Failed! Total available frames (5) < Required minimum frames (6).
```

```
Roll no: 25071A6255

Enter total frames in physical memory: 12
Enter minimum frames required per process (by architecture): 2
Enter number of processes: 3

Enter page requirements for each process:
Process P1: 2
Process P2: 2
Process P3: 2

--- Minimum Frame Allocation Result ---
Process Page Req        Allocated Frames        Status
P1      2               2                       Satisfied
P2      2               2                       Satisfied
P3      2               2                       Satisfied

Total Frames Allocated: 6 / 12
Remaining Unallocated Frames: 6
```

B. equal allocation

Program:

```
#include <stdio.h>

int main() {
    printf("Roll no: 25071A6255\n\n");
    int total_frames, m;
    int page_req[10], allocated[10];

    printf("Enter total frames in physical memory: ");
    scanf("%d", &total_frames);
    printf("Enter number of processes: ");
    scanf("%d", &m);

    if (m <= 0 || total_frames < m) {
        printf("\nAllocation Failed! Insufficient frames for equal allocation.\n");
        return 0;
    }

    printf("Enter page requirements for each process:\n");
    for (int i = 0; i < m; i++) {
        printf("Process P%d: ", i + 1);
        scanf("%d", &page_req[i]);
    }

    int equal_share = total_frames / m;
    int leftover = total_frames % m;

    printf("\n--- Equal Frame Allocation Result ---\n");
    printf("Process\tPage Req\tAllocated Frames\tUnmet Pages\n");
    for (int i = 0; i < m; i++) {
        allocated[i] = equal_share;
        int unmet = (page_req[i] > allocated[i]) ? (page_req[i] - allocated[i]) : 0;
        printf("P%d\t%d\t\t%d\t\t\t%d\n", i + 1, page_req[i], allocated[i], unmet);
    }

    printf("\nTotal Allocated Frames: %d / %d", equal_share * m, total_frames);
    printf("\nLeftover System Frames: %d\n", leftover);

    return 0;
}
```

Testcases:

```
Roll no: 25071A6255

Enter total frames in physical memory: 63
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 10
Process P2: 127
Process P3: 18

--- Equal Frame Allocation Result ---
Process Page Req        Allocated Frames        Unmet Pages
P1      10              21                      0
P2      127             21                      106
P3      18              21                      0

Total Allocated Frames: 63 / 63
Leftover System Frames: 0
```

```
Roll no: 25071A6255

Enter total frames in physical memory: 100
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 40
Process P2: 25
Process P3: 60

--- Equal Frame Allocation Result ---
Process Page Req        Allocated Frames        Unmet Pages
P1      40              33                      7
P2      25              33                      0
P3      60              33                      27

Total Allocated Frames: 99 / 100
Leftover System Frames: 1
```

```
Roll no: 25071A6255

Enter total frames in physical memory: 60
Enter number of processes: 3
Enter page requirements for each process:
Process P1: 15
Process P2: 12
Process P3: 18

--- Equal Frame Allocation Result ---
Process Page Req        Allocated Frames        Unmet Pages
P1      15              20                      0
P2      12              20                      0
P3      18              20                      0

Total Allocated Frames: 60 / 60
Leftover System Frames: 0
```

C. proportional allocation

Program:

```
#include <stdio.h>
int main() {
   printf("Roll no: 25071A6255\n\n");
   int m, n;
   int s[10], a[10];
   int S = 0, total_allocated = 0;

   printf("Enter total frames available in physical memory: ");
   scanf("%d", &m);
   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter size (in pages/virtual memory) for each process:\n");
   for (int i = 0; i < n; i++) {
      printf("Process P%d size: ", i + 1);
      scanf("%d", &s[i]);
      S += s[i];
   }

   if (S == 0) {
      printf("\nInvalid total process size.\n");
      return 0;
   }
   printf("\n--- Proportional Frame Allocation Result ---\n");
   printf("Total Virtual Memory Required (S) = %d pages\n\n", S);
   printf("Process\tSize (s_i)\tAllocated Frames (a_i)\tPercentage\n");

   for (int i = 0; i < n; i++) {
      a[i] = (int)(((double)s[i] / S) * m);
      total_allocated += a[i];
      double pct = ((double)s[i] / S) * 100.0;
      printf("P%d\t%d\t\t%d\t\t\t%.2f%%\n", i + 1, s[i], a[i], pct);
   }
   printf("\nTotal Frames Allocated: %d / %d", total_allocated, m);
   printf("\nUnallocated Buffer Frames: %d\n", m - total_allocated);

   return 0;
}
```

Testcases:

```
Roll no: 25071A6255

Enter total frames available in physical memory: 62
Enter number of processes: 2
Enter size (in pages/virtual memory) for each process:
Process P1 size: 10
Process P2 size: 127

--- Proportional Frame Allocation Result ---
Total Virtual Memory Required (S) = 137 pages

Process Size (s_i)      Allocated Frames (a_i)  Percentage
P1      10              4                       7.30%
P2      127             57                      92.70%

Total Frames Allocated: 61 / 62
Unallocated Buffer Frames: 1
```

```
Roll no: 25071A6255

Enter total frames available in physical memory: 100
Enter number of processes: 3
Enter size (in pages/virtual memory) for each process:
Process P1 size: 50
Process P2 size: 50
Process P3 size: 50

--- Proportional Frame Allocation Result ---
Total Virtual Memory Required (S) = 150 pages

Process Size (s_i)      Allocated Frames (a_i)  Percentage
P1      50              33                      33.33%
P2      50              33                      33.33%
P3      50              33                      33.33%

Total Frames Allocated: 99 / 100
Unallocated Buffer Frames: 1
```

```
Roll no: 25071A6255

Enter total frames available in physical memory: 60
Enter number of processes: 3
Enter size (in pages/virtual memory) for each process:
Process P1 size: 20
Process P2 size: 30
Process P3 size: 50

--- Proportional Frame Allocation Result ---
Total Virtual Memory Required (S) = 100 pages

Process Size (s_i)      Allocated Frames (a_i)  Percentage
P1      20              12                      20.00%
P2      30              18                      30.00%
P3      50              30                      50.00%

Total Frames Allocated: 60 / 60
Unallocated Buffer Frames: 0
```

Aim: Simulate the following Disk scheduling algorithms. A) FCFS B) SSTF C) SCAN

A) FCFS

Algorithm:

1. Start.
2. Input the number of disk requests (n), request sequence, and initial head position.
3. Initialize Total Head Movement (THM) = 0.
4. For each request:
   - Calculate the distance between the current head and the request.
   - Add the distance to THM.
   - Move the head to the requested position.
5. Calculate:
   - Average Seek Time = THM / n
6. Display the Total Head Movement and Average Seek Time.
7. Stop.

B) SSTF

Algorithm:

1. **Start**
2. Input number of requests n, request sequence, and initial head position.
3. Initialize THM = 0 and mark all requests as unvisited.
4. Repeat n times:
   - Find the unvisited request closest to the current head.
   - Mark it as visited.
   - Add the distance to THM.
   - Move the head to that request.
5. Calculate:
   - Average Seek Time = THM / n
6. Display Total Head Movement and Average Seek Time.
7. **Stop**.

c) SCAN

Algorithm:

1. **Start**.
2. Input number of requests n, request sequence, initial head position, disk size, and direction (LEFT or RIGHT).
3. Sort the request sequence in ascending order.
4. Initialize THM = 0 and divide requests based on the current head position.
5. If direction is RIGHT:
6. Service requests to the right in ascending order.
7. Move the head to disk_size - 1.
8. Reverse direction and service remaining requests in descending order.
9. If direction is LEFT:
10. Service requests to the left in descending order.
11. Move the head to 0.
12. Reverse direction and service remaining requests in ascending order.
13. Calculate Average Seek Time = THM / n and display Total Head Movement and Average Seek Time.
14. **Stop.**

A) FCFS

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, head, req[20], thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);

   printf("\nSeek Sequence: %d", head);
   for (int i = 0; i < n; i++) {
      thm += abs(req[i] - head);
      head = req[i];
      printf(" -> %d", head);
   }
   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / n);
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Seek Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Head Movement = 640
Average Seek Time = 80.00
```

```
Roll no: 25071A6255
Enter number of requests: 7
Enter request sequence: 82 170 43 140 24 16 190
Enter initial head position: 50

Seek Sequence: 50 -> 82 -> 170 -> 43 -> 140 -> 24 -> 16 -> 190
Total Head Movement = 642
Average Seek Time = 91.71
```

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request sequence: 176 79 34 60 92 11 41 114
Enter initial head position: 50

Seek Sequence: 50 -> 176 -> 79 -> 34 -> 60 -> 92 -> 11 -> 41 -> 114
Total Head Movement = 510
Average Seek Time = 63.75
```

B) SSTF

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    printf("Roll no: 25071A6255\n");
    int n, head, req[20], visited[20] = {0}, thm = 0;
    printf("Enter number of requests: "); scanf("%d", &n);
    printf("Enter request sequence: ");
    for (int i = 0; i < n; i++) scanf("%d", &req[i]);
    printf("Enter initial head position: "); scanf("%d", &head);

    printf("\nSeek Sequence: %d", head);
    for (int count = 0; count < n; count++) {
        int min_dist = 1e9, idx = -1;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && abs(req[i] - head) < min_dist) {
                min_dist = abs(req[i] - head);
                idx = i;
            }
        }
        visited[idx] = 1;
        thm += min_dist;
        head = req[idx];
        printf(" -> %d", head);
    }
    printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / n);
    return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request sequence: 176 79 34 60 92 11 41 114
Enter initial head position: 50

Seek Sequence: 50 -> 41 -> 34 -> 11 -> 60 -> 79 -> 92 -> 114 -> 176
Total Head Movement = 204
Average Seek Time = 25.50
```

```
Roll no: 25071A6255
Enter number of requests: 7
Enter request sequence: 82 170 43 140 24 16 190
Enter initial head position: 50

Seek Sequence: 50 -> 43 -> 24 -> 16 -> 82 -> 140 -> 170 -> 190
Total Head Movement = 208
Average Seek Time = 29.71
```

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Seek Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Head Movement = 236
Average Seek Time = 29.50
```

C) SCAN

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, h, r[30], s, d, thm = 0, p = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: "); for (int i = 0; i < n; i++) scanf("%d", &r[i]);
   printf("Enter initial head position: "); scanf("%d", &h);
   printf("Enter total disk size: "); scanf("%d", &s);
   printf("Enter direction (1 for High/Right, 0 for Low/Left): "); scanf("%d", &d);
   r[n++] = h; r[n++] = d ? s - 1 : 0;
   for (int i = 0; i < n - 1; i++)
      for (int j = 0; j < n - i - 1; j++)
         if (r[j] > r[j + 1]) { int t = r[j]; r[j] = r[j + 1]; r[j + 1] = t; }
   while (r[p] != h) p++;
   printf("\nSeek Sequence: %d", h);
   int cur = h, step = d ? 1 : -1, start = d ? p + 1 : p - 1, end = d ? n : -1;
   for (int pass = 0; pass < 2; pass++) {
      for (int i = start; i != end; i += step) { thm += abs(r[i] - cur); cur = r[i]; printf(" -> %d", cur); }
      step = -step; start = (d == (pass == 0)) ? p - 1 : p + 1; end = (d == (pass == 0)) ? -1 : n;
   }
   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 2));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request sequence: 98 183 37 122 14 124 65 67
Enter initial head position: 53
Enter total disk size: 200
Enter direction (1 for High/Right, 0 for Low/Left): 1

Seek Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199 -> 37 -> 14
Total Head Movement = 331
Average Seek Time = 41.38
```

```
Roll no: 25071A6255
Enter number of requests: 7
Enter request sequence: 82 170 43 140 24 16 190
Enter initial head position: 50
Enter total disk size: 200
Enter direction (1 for High/Right, 0 for Low/Left): 1

Seek Sequence: 50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 43 -> 24 -> 16
Total Head Movement = 332
Average Seek Time = 47.43
```

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request sequence: 176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter total disk size: 200
Enter direction (1 for High/Right, 0 for Low/Left): 0

Seek Sequence: 50 -> 41 -> 34 -> 11 -> 0 -> 60 -> 79 -> 92 -> 114 -> 176
Total Head Movement = 226
Average Seek Time = 28.25
```

VNRVJIET

Name of the laboratory:
OPERATING SYSTEMS

Name of the Experiment: **Disk scheduling Algorithms a) C-SCAN b) LOOK c) C-LOOK**

Experiment No: 9 **Date:** **17-09-2026**

Aim: Simulate the following Disk scheduling algorithms. A) C-SCAN B) LOOK C) C-LOOK

A) C- SCAN

Algorithm:

1. Start.
2. Input number of requests n, request sequence, initial head position, and disk size.
3. Sort the request sequence in ascending order.
4. If direction is RIGHT:
    - Service requests from the current head towards disk_size - 1.
    - Move the head directly to 0.
    - Continue servicing the remaining requests in ascending order.
5. Calculate Total Head Movement and Average Seek Time.
6. Display the results.
7. Stop.

B) LOOK

Algorithm:

1. **Start.**
2. Input number of requests n, request sequence, initial head position, and direction.
3. Sort the request sequence in ascending order.
4. If direction is RIGHT:
    - Service requests to the right up to the maximum requested track.
    - Reverse direction and service the remaining requests to the left.
5. If direction is LEFT:
    - Service requests to the left up to the minimum requested track.
    - Reverse direction and service the remaining requests to the right.
6. Calculate Total Head Movement and Average Seek Time.
7. Display the results.
8. **Stop.**

B) C-LOOK

Algorithm:

1. **Start**.
2. Input number of requests n, request sequence, initial head position, and direction.
3. Sort the request sequence in ascending order.
4. If direction is RIGHT:
    - Service requests to the right up to the maximum requested track.
    - Jump directly to the minimum requested track.
    - Service the remaining requests up to the track before the starting head.
5. Calculate Total Head Movement and Average Seek Time.
6. Display the results.
7. **Stop.**

A) C-SCAN

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, head, req[30], disk_size, thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);
   printf("Enter total disk size: "); scanf("%d", &disk_size);

   req[n++] = head;
   req[n++] = 0;
   req[n++] = disk_size - 1;

   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (req[j] > req[j + 1]) {
            int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t;}}
     }
   int pos = 0;
   for (int i = 0; i < n; i++) if (req[i] == head) { pos = i; break; }

   printf("\nSeek Sequence: %d", head);
   int cur = head;
   for (int i = pos + 1; i < n; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
  }
   for (int i = 0; i < pos; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);}
   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 3));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Sequence of head movement:
65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14 -> 37 ->
Total Seek Time = 382
Average Seek Time = 47.75
```

```
Roll no: 25071A6255
Enter number of requests: 6
Enter request queue: 10 150 20 140 30 130
Enter initial head position: 100

Sequence of head movement:
130 -> 140 -> 150 -> 10 -> 20 -> 30 ->
Total Seek Time = 328
Average Seek Time = 54.67
```

```
Roll no: 25071A6255
Enter number of requests: 5
Enter request queue: 45 12 67 85 123
Enter initial head position: 50

Sequence of head movement:
67 -> 85 -> 123 -> 12 -> 45 ->
Total Seek Time = 393
Average Seek Time = 78.60
```

B) LOOK

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, head, req[25], thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);

   req[n++] = head;
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (req[j] > req[j + 1]) {
            int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t;
         }
      }
   }

   int pos = 0;
   for (int i = 0; i < n; i++) if (req[i] == head) { pos = i; break; }

   printf("\nSeek Sequence: %d", head);
   int cur = head;
   for (int i = pos + 1; i < n; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }
   for (int i = pos - 1; i >= 0; i--) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }

   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 1));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request queue: 98 183 37 122 14 124 65 67
Enter initial head position: 53

Sequence of head movement:
65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 37 -> 14 ->
Total Seek Time = 299
Average Seek Time = 37.38
```

```
Roll no: 25071A6255
Enter number of requests: 5
Enter request queue: 176 79 34 60 92
Enter initial head position: 50

Sequence of head movement:
60 -> 79 -> 92 -> 176 -> 34 ->
Total Seek Time = 268
Average Seek Time = 53.60
```

```
Roll no: 25071A6255
Enter number of requests: 6
Enter request queue: 10 150 20 140 30 130
Enter initial head position: 100

Sequence of head movement:
130 -> 140 -> 150 -> 30 -> 20 -> 10 ->
Total Seek Time = 190
Average Seek Time = 31.67
```

C) C-LOOK

Program:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
   printf("Roll no: 25071A6255\n");
   int n, head, req[25], thm = 0;
   printf("Enter number of requests: "); scanf("%d", &n);
   printf("Enter request sequence: ");
   for (int i = 0; i < n; i++) scanf("%d", &req[i]);
   printf("Enter initial head position: "); scanf("%d", &head);

   req[n++] = head;
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (req[j] > req[j + 1]) {
            int t = req[j]; req[j] = req[j + 1]; req[j + 1] = t;
         }
      }
   }

   int pos = 0;
   for (int i = 0; i < n; i++) if (req[i] == head) { pos = i; break; }

   printf("\nSeek Sequence: %d", head);
   int cur = head;
   for (int i = pos + 1; i < n; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }
   for (int i = 0; i < pos; i++) {
      thm += abs(req[i] - cur); cur = req[i]; printf(" -> %d", cur);
   }

   printf("\nTotal Head Movement = %d\nAverage Seek Time = %.2f\n", thm, (float)thm / (n - 1));
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of requests: 5
Enter request queue: 176 79 34 60 92
Enter initial head position: 50

Sequence of head movement:
60 -> 79 -> 92 -> 176 -> 34 ->
Total Seek Time = 268
Average Seek Time = 53.60
```

```
Roll no: 25071A6255
Enter number of requests: 8
Enter request queue: 98 193 37 122 14 124 65 67
Enter initial head position: 53

Sequence of head movement:
65 -> 67 -> 98 -> 122 -> 124 -> 193 -> 14 -> 37 ->
Total Seek Time = 342
Average Seek Time = 42.75
```

```
Roll no: 25071A6255
Enter number of requests: 6
Enter request queue: 10 150 20 140 30 130
Enter initial head position: 100

Sequence of head movement:
130 -> 140 -> 150 -> 10 -> 20 -> 30 ->
Total Seek Time = 210
Average Seek Time = 35.00
```

Aim: Simulate all file allocation strategies. A. Sequential, B. Indexed, C. Linked

A. Sequential

Algorithm:

1. **Start**.
2. Input disk block status, total number of blocks, starting block, and file length.
3. Check whether all blocks from start to start + length - 1 are free.
4. If all blocks are free, allocate them to the file.
5. Display the allocation status and allocated block numbers.
6. **Stop**.

B. Indexed

Algorithm:

1. **Start**.
2. Input index block number and number of file blocks n.
3. Check whether the index block is free.
4. Input the required block numbers and check whether all blocks are free.
5. If all blocks are free, allocate the index block and all required blocks.
6. Display the index block and its allocated block mapping.
7. **Stop**.

C. Linked

Algorithm:

1. **Start**.
2. Input starting block number and number of blocks n.
3. Input the block numbers forming the linked chain.
4. Check whether all required blocks are free.
5. Allocate the blocks and link them in sequence, ending with -1.
6. Display the linked block sequence.
7. **Stop**.

A. Sequential

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A6255\n");
   int disk[50] = {0}, start, len, alloc = 1;
   printf("Enter start block and length of file: ");
   scanf("%d %d", &start, &len);
   for (int i = start; i < start + len; i++) if (disk[i]) alloc = 0;
   if (alloc) {
      printf("File allocated successfully. Blocks: ");
      for (int i = start; i < start + len; i++) {
         disk[i] = 1; printf("%d ", i);
      }
      printf("\n");
   } else printf("Allocation failed: Contiguous blocks unavailable.\n");
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter start block and length of file: 12 4
File allocated successfully. Blocks: 12 13 14 15
```

```
Roll no: 25071A6255
Enter start block and length of file: 0 5
File allocated successfully. Blocks: 0 1 2 3 4
```

```
Roll no: 25071A6255
Enter start block and length of file: 20 6
File allocated successfully. Blocks: 20 21 22 23 24 25
```

B. Indexed

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A6255\n");
   int disk[50] = {0}, idx, n, b[20], alloc = 1;
   printf("Enter index block: "); scanf("%d", &idx);
   printf("Enter number of blocks and block numbers: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) scanf("%d", &b[i]);
   if (disk[idx]) alloc = 0;
   for (int i = 0; i < n; i++) if (disk[b[i]]) alloc = 0;
   if (alloc) {
      disk[idx] = 1; printf("Index Block %d points to: ", idx);
      for (int i = 0; i < n; i++) { disk[b[i]] = 1; printf("%d ", b[i]); }
      printf("\nFile allocated successfully.\n");
   } else printf("Allocation failed: Index or data blocks already allocated.\n");
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter index block: 9
Enter number of blocks and block numbers: 4 14 18 23 29
Index Block 9 points to: 14 18 23 29
File allocated successfully.
```

```
Roll no: 25071A6255
Enter index block: 5
Enter number of blocks and block numbers: 3 10 20 30
Index Block 5 points to: 10 20 30
File allocated successfully.
```

```
Roll no: 25071A6255
Enter index block: 9
Enter number of blocks and block numbers: 3 4 11 15
Index Block 9 points to: 4 11 15
File allocated successfully.
```

C. Linked

Program:

```
#include <stdio.h>

int main() {
   printf("Roll no: 25071A6255\n");
   int disk[50] = {0}, n, b[20], alloc = 1;
   printf("Enter number of blocks and block sequence: "); scanf("%d", &n);
   for (int i = 0; i < n; i++) scanf("%d", &b[i]);
   for (int i = 0; i < n; i++) if (disk[b[i]]) alloc = 0;
   if (alloc) {
      printf("Linked Allocation Sequence: ");
      for (int i = 0; i < n; i++) {
         disk[b[i]] = 1; printf("%d -> ", b[i]);
      }
      printf("NULL\nFile allocated successfully.\n");
   } else printf("Allocation failed: One or more blocks already allocated.\n");
   return 0;
}
```

Testcases:

```
Roll no: 25071A6255
Enter number of blocks and block sequence: 4 5 11 25 34
Linked Allocation Sequence: 5 -> 11 -> 25 -> 34 -> NULL
File allocated successfully.
```

```
Roll no: 25071A6255
Enter number of blocks and block sequence: 3 2 8 18
Linked Allocation Sequence: 2 -> 8 -> 18 -> NULL
File allocated successfully.
```

```
Roll no: 25071A6255
Enter number of blocks and block sequence: 4 1 5 12 20
Linked Allocation Sequence: 1 -> 5 -> 12 -> 20 -> NULL
File allocated successfully.
```